

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, mean_squared_error, accuracy
from sklearn.preprocessing import OrdinalEncoder, StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
```

✓ Understanding the Data

Start coding or [generate](#) with AI.

✓ Read in the Data

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount('/content/drive', force_remount=True)`

```
# 1. Load data
url = "https://health.data.ny.gov/resource/tg3i-cinn.csv"
df = pd.read_csv(url)
```

```
# 2. Preview / basic info
df.head()
```

	hospital_service_area	hospital_county	operating_certificate_number	permane
0	New York City	Bronx	7000006.0	
1	New York City	Bronx	7000006.0	
2	New York City	Bronx	7000006.0	
3	New York City	Bronx	7000006.0	
4	New York City	Bronx	7000006.0	

5 rows × 33 columns

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   hospital_service_area                995 non-null    object
1   hospital_county                      995 non-null    object
2   operating_certificate_number          995 non-null    float64
3   permanent_facility_id                995 non-null    float64
4   facility_name                        1000 non-null   object
5   age_group                            1000 non-null   object
6   zip_code_3_digits                    977 non-null    object
7   gender                               1000 non-null   object
8   race                                 1000 non-null   object
9   ethnicity                            1000 non-null   object
10  length_of_stay                       1000 non-null   object
11  type_of_admission                    1000 non-null   object
12  patient_disposition                  1000 non-null   object
13  discharge_year                       1000 non-null   int64
14  ccsr_diagnosis_code                  1000 non-null   object
15  ccsr_diagnosis_description            1000 non-null   object
16  ccsr_procedure_code                   755 non-null    object
17  ccsr_procedure_description            755 non-null    object
```

```

18 apr_drg_code          1000 non-null    int64
19 apr_drg_description   1000 non-null    object
20 apr_mdc_code          1000 non-null    int64
21 apr_mdc_description   1000 non-null    object
22 apr_severity_of_illness_code 1000 non-null    int64
23 apr_severity_of_illness 1000 non-null    object
24 apr_risk_of_mortality  1000 non-null    object
25 apr_medical_surgical  1000 non-null    object
26 payment_typology_1    1000 non-null    object
27 payment_typology_2    252 non-null     object
28 payment_typology_3    40 non-null      object
29 birth_weight          188 non-null     float64
30 emergency_department_indicator 1000 non-null    object
31 total_charges         1000 non-null    float64
32 total_costs           1000 non-null    float64
dtypes: float64(5), int64(4), object(24)
memory usage: 257.9+ KB

```

```
df.describe()
```

	operating_certificate_number	permanent_facility_id	discharge_year	apr_
count	9.950000e+02	995.000000	1000.0	10
mean	6.939185e+06	1461.255276	2021.0	5
std	4.024881e+05	487.682229	0.0	2
min	4.324000e+06	776.000000	2021.0	
25%	7.000008e+06	1172.000000	2021.0	2
50%	7.001016e+06	1301.000000	2021.0	5
75%	7.002001e+06	1456.000000	2021.0	7
max	7.003007e+06	3058.000000	2021.0	9

✓ Understanding the Features/Columns

1. Patient Demographics

- **age_group** → Age ranges (e.g., 0–17, 18–29, 30–49, 50–69, 70+) Useful for grouping patients by age category.
- **gender** → Male/Female/Other. **race** → Race categories (White, Black, Asian/Pacific Islander, Other).
- **ethnicity** → Hispanic / Non-Hispanic.
- **zip_code_3_digits** → First 3 digits of patient ZIP (for geography).

2. Admission & Discharge Information

- `type_of_admission` → How patient was admitted: Emergency, Elective, Trauma, Newborn.
- `patient_disposition` → Status at discharge: home, rehab, skilled nursing, expired, transferred.
- `emergency_department_indicator` → Y/N if admitted via ED.
- `discharge_year` → Year of discharge (time-based trends).

3. Clinical Classifications

- `ccsr_diagnosis_code` / `ccsr_diagnosis_description` → Diagnosis category (based on ICD → CCSR grouping).
- `ccsr_procedure_code` / `ccsr_procedure_description` → Procedure category.
- `apr_drg_code` / `apr_drg_description` → All-Patient Refined DRG (Diagnosis Related Group).
- `apr_mdc_code` / `apr_mdc_description` → Major Diagnostic Category.
- `apr_severity_of_illness_code` / `apr_severity_of_illness` → Severity level (1=Minor, 2=Moderate, 3=Major, 4=Extreme).
- `apr_risk_of_mortality` → Mortality risk category (Minor, Moderate, Major, Extreme).
- `apr_medical_surgical` → Medical vs Surgical case.

4. Hospital Identifiers

- `hospital_service_area` → Regional service area.
- `hospital_county` → County where hospital is located.
- `facility_name` → Hospital facility name.
- `operating_certificate_number`, `permanent_facility_id` → Hospital identifiers.

5. Financial Variables

- `payment_typology_1` / `2` / `3` → Primary, secondary, tertiary payer (Medicare, Medicaid, private insurance, self-pay).
- `total_charges` → Amount billed to patient/insurer.
- `total_costs` → Estimated hospital cost of the stay.

6. Outcome Variable

- `length_of_stay` → Number of days patient stayed in hospital (integer, sometimes capped as "120+").
- Target for regression.
- Distribution is usually right-skewed (most short stays, few very long ones).

✓ Dealing with missing values and One-hot encoding

```
#checking shape before modifying data  
df.shape
```

```
(1000, 33)
```

```
# 3. Check missing values  
df.isnull().sum().sort_values(ascending=False)
```


	0
payment_typology_3	960
birth_weight	812
payment_typology_2	748

```
#drop duplicates
df.drop_duplicates(inplace=True)
```

```
# Drop unnecessary columns
df = df.drop(columns=['operating_certificate_number', 'permanent_facility_id', 'f
                    'payment_typology_2', 'payment_typology_3', 'birth_weight',
```

```
# First, convert everything to string so .str methods work
df['length_of_stay'] = df['length_of_stay'].astype(str)

# Remove "+" signs if they exist (e.g., "120+")
df['length_of_stay'] = df['length_of_stay'].str.replace('+', '', regex=False)

# Convert back to numeric
df['length_of_stay'] = pd.to_numeric(df['length_of_stay'], errors='coerce')
```

	facility_name	0
--	---------------	---

```
print(df['length_of_stay'].describe())
```

count	999	age_group	0
mean	6.491491		
std	10.765567	ccsr_diagnosis_description	0
min	1.000000		
25%	2.000000	ccsr_diagnosis_code	0
50%	3.000000		
75%	7.000000	discharge_year	0
max	120.000000		
Name: length_of_stay, dtype: float64		patient_disposition	0
		apr_mdc_description	0

```
df.isnull().sum().sort_values(ascending=False)
```

apr_drg_description	0
apr_mdc_code	0
apr_risk_of_mortality	0
apr_severity_of_illness	0
apr_severity_of_illness_code	0
payment_typology_1	0
apr_medical_surgical	0

emergency_department_indicator	0
hospital_service_area	5
total_costs	0
hospital_county	5
age_group	0
gender	0
race	0
ethnicity	0
length_of_stay	0
type_of_admission	0
patient_disposition	0
ccsr_diagnosis_code	0
ccsr_diagnosis_description	0
apr_drg_code	0
apr_drg_description	0
apr_mdc_code	0
apr_mdc_description	0
apr_severity_of_illness_code	0
apr_severity_of_illness	0
apr_risk_of_mortality	0
apr_medical_surgical	0
payment_typology_1	0
emergency_department_indicator	0

dtype: int64

```
print(df.shape)
```

```
(999, 21)
```

Several columns were removed from the dataset because they were either identifiers, post-discharge variables, or provided little predictive value. Hospital identifiers (operating_certificate_number, permanent_facility_id, facility_name) were dropped since they don't describe patient or clinical characteristics. Financial outcomes (total_charges, total_costs) were excluded to avoid data leakage, as they are only known after

discharge. Secondary/tertiary payment fields, birth weight, and partial ZIP codes were removed due to high missingness, sparsity, or limited relevance. Finally, CCSR procedure codes and descriptions were dropped because they overlap with existing diagnosis/severity fields and add redundant complexity. This ensures the model uses only clinically meaningful, admission-time features.

✓ Outlier Analysis

```
# Summary statistics
print(df['length_of_stay'].describe())

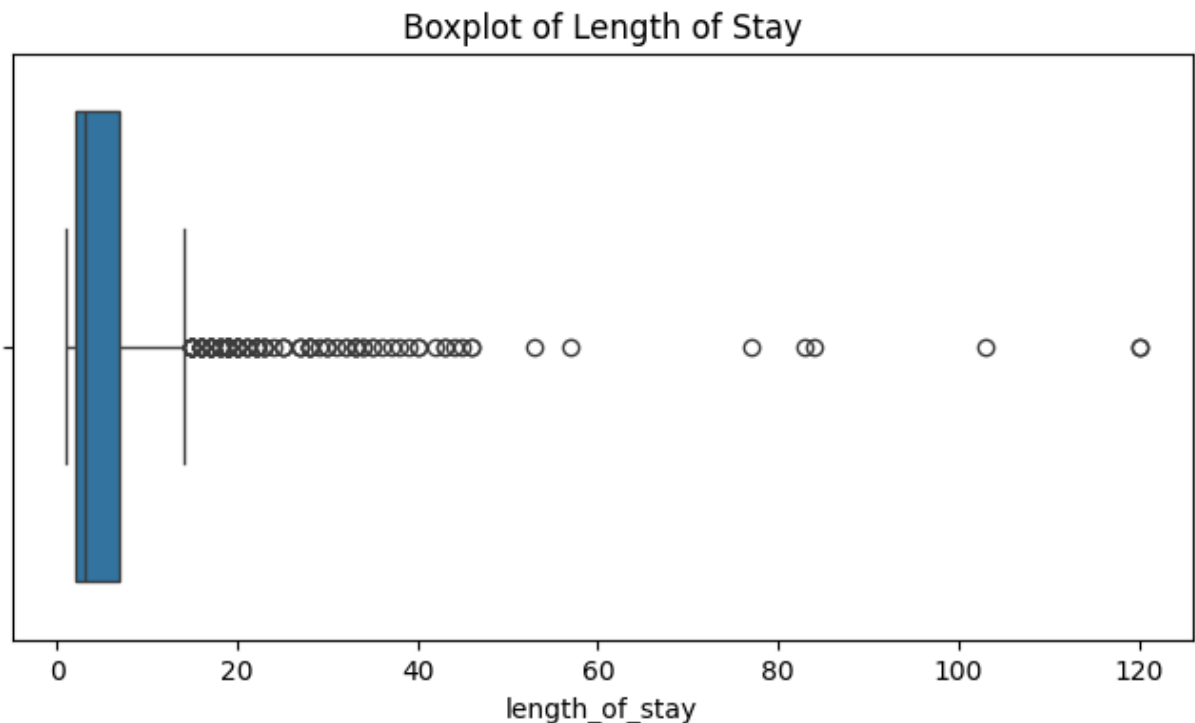
# Boxplot to visualize outliers
plt.figure(figsize=(8,4))
sns.boxplot(x=df['length_of_stay'])
plt.title("Boxplot of Length of Stay")
plt.show()

# Count extreme long stays (e.g., >30 days)
outliers = df[df['length_of_stay'] > 30]
print(f"Number of stays >30 days: {len(outliers)}")
print(outliers['length_of_stay'].value_counts().sort_index().tail(10))
```

```

count      999.000000
mean        6.491491
std         10.705367
min          1.000000
25%          2.000000
50%          3.000000
75%          7.000000
max        120.000000
Name: length_of_stay, dtype: float64

```



```

Number of stays >30 days: 32
length_of_stay
44      1
45      1
46      2
53      1
57      1
77      1
83      1
84      1
103     1
120     2
Name: count, dtype: int64

```

Outlier analysis of the target variable (`length_of_stay`) shows a highly skewed distribution with most patients staying under 7 days, but a small fraction of admissions extending beyond 30 days (up to 120). These extreme cases are clinically valid and represent high-severity conditions rather than errors. For this project, we retain all values but apply a log transformation during modeling to reduce skewness and lessen the impact of extreme outliers on model performance.

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Log transform (adding 1 so zero stays valid)
df['log_los'] = np.log1p(df['length_of_stay'])

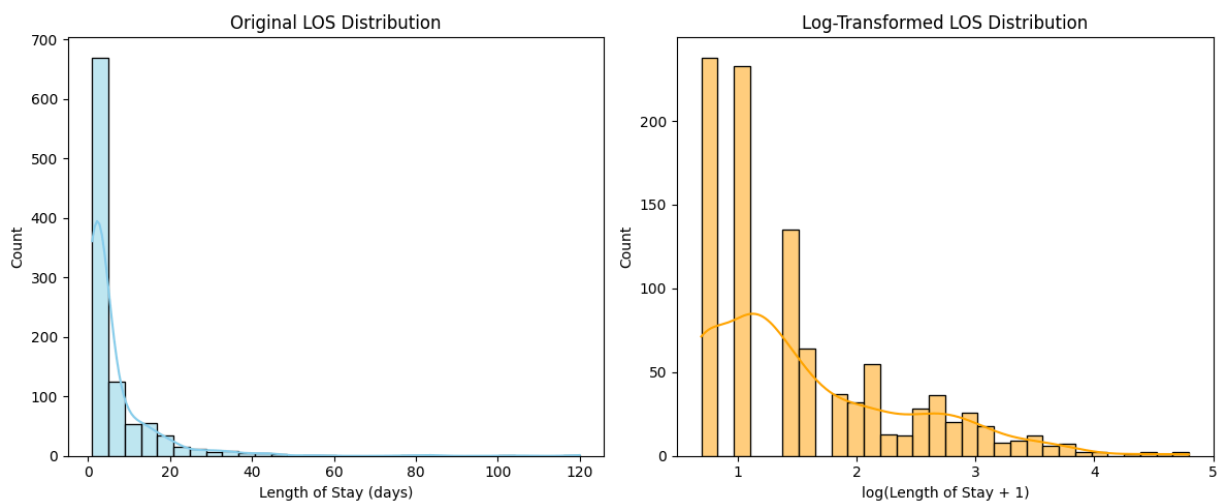
# Plot original vs log-transformed side by side
fig, axes = plt.subplots(1, 2, figsize=(12,5))

sns.histplot(df['length_of_stay'], bins=30, kde=True, ax=axes[0], color="skyblue")
axes[0].set_title("Original LOS Distribution")
axes[0].set_xlabel("Length of Stay (days)")

sns.histplot(df['log_los'], bins=30, kde=True, ax=axes[1], color="orange")
axes[1].set_title("Log-Transformed LOS Distribution")
axes[1].set_xlabel("log(Length of Stay + 1)")

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Colab Notebooks/Module 20 : Ensemble Techni
plt.show()

```

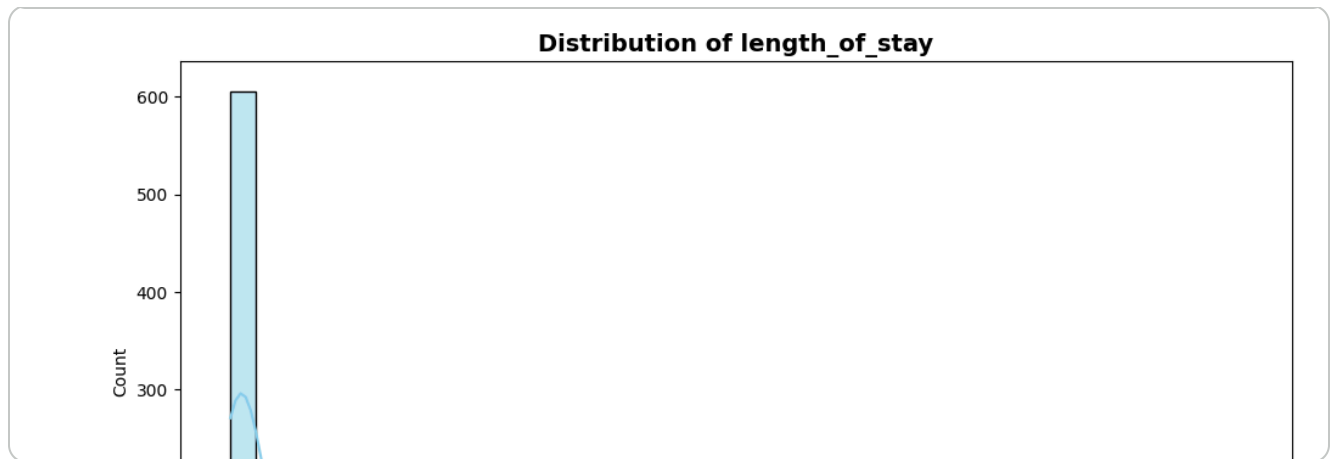


✓ Exploratory Data Analysis

Univariate Numerical Analysis

```
# Select numerical columns (excluding target if desired)
numerical_cols = ['length_of_stay', 'apr_severity_of_illness_code']

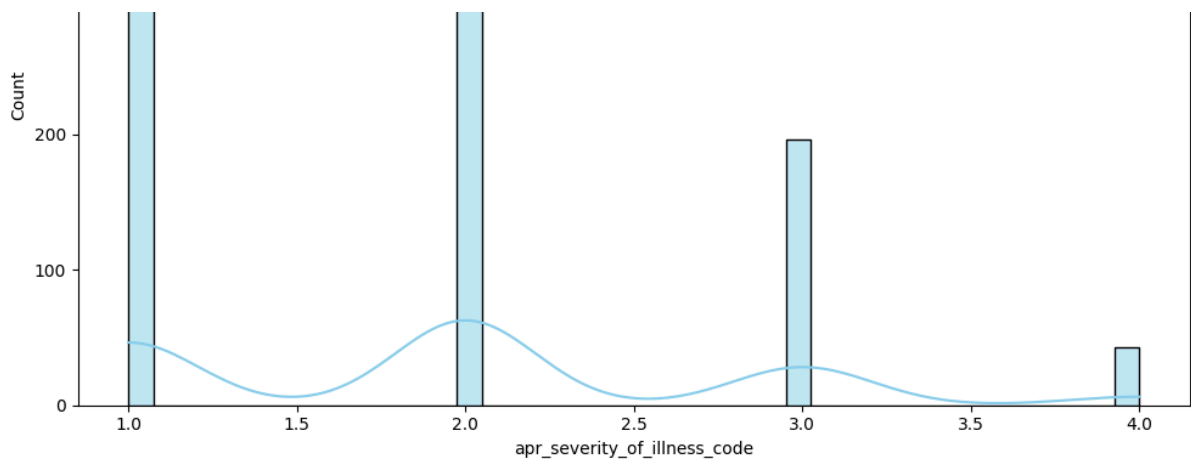
# Plot distribution for each numerical feature
fig, axes = plt.subplots(nrows=len(numerical_cols), ncols=1, figsize=(10, 6 * 1
for ax, col in zip(axes, numerical_cols):
    sns.histplot(df[col].dropna(), bins=40, kde=True, color='skyblue', ax=ax)
    ax.set_title(f'Distribution of {col}', fontsize=14, fontweight='bold')
    ax.set_xlabel(col)
    ax.set_ylabel('Count')
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Colab Notebooks/Module 20 : Ensemble Techni
plt.show()
```

Univariate Categorical Analysis

```
cat_cols = ['type_of_admission', 'apr_severity_of_illness', 'apr_risk_of_mortalit
fig, axes = plt.subplots(nrows=len(cat_cols), ncols=1, figsize=(10, 5 * len(cat
for ax, col in zip(axes, cat_cols):
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
    ax.set_title(f"Distribution of {col}")
```

```
plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Colab Notebooks/Module 20 : Ensemble Techni
plt.show()
```




```
/tmp/ipython-input-19261604.py:4: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
/tmp/ipython-input-19261604.py:4: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
/tmp/ipython-input-19261604.py:4: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
/tmp/ipython-input-19261604.py:4: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
/tmp/ipython-input-19261604.py:4: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
/tmp/ipython-input-19261604.py:4: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
/tmp/ipython-input-19261604.py:4: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
/tmp/ipython-input-19261604.py:4: FutureWarning:
```

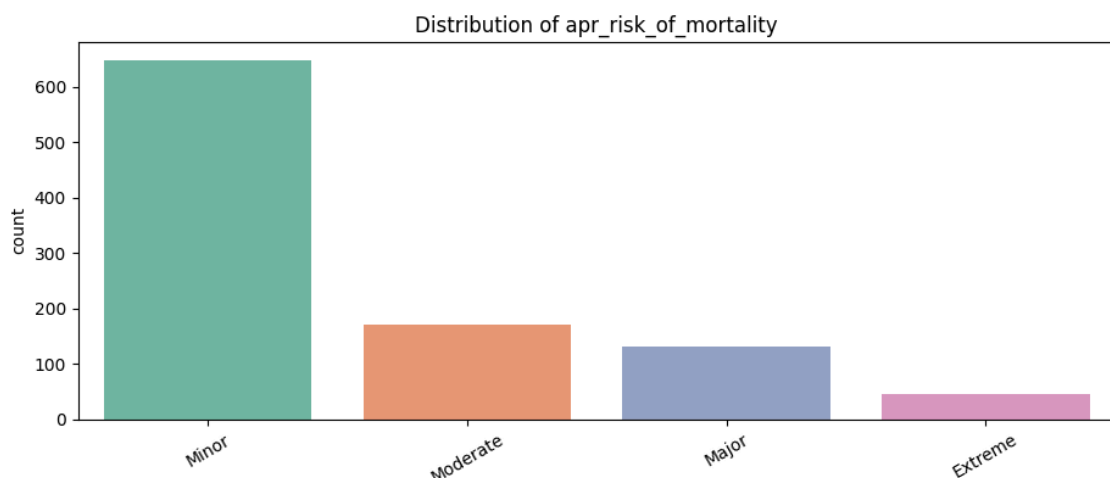
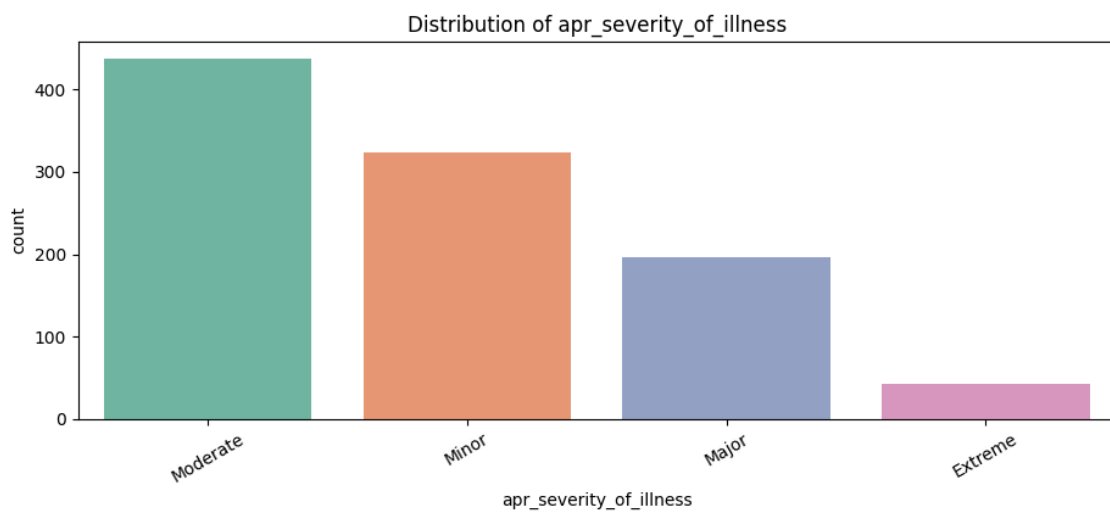
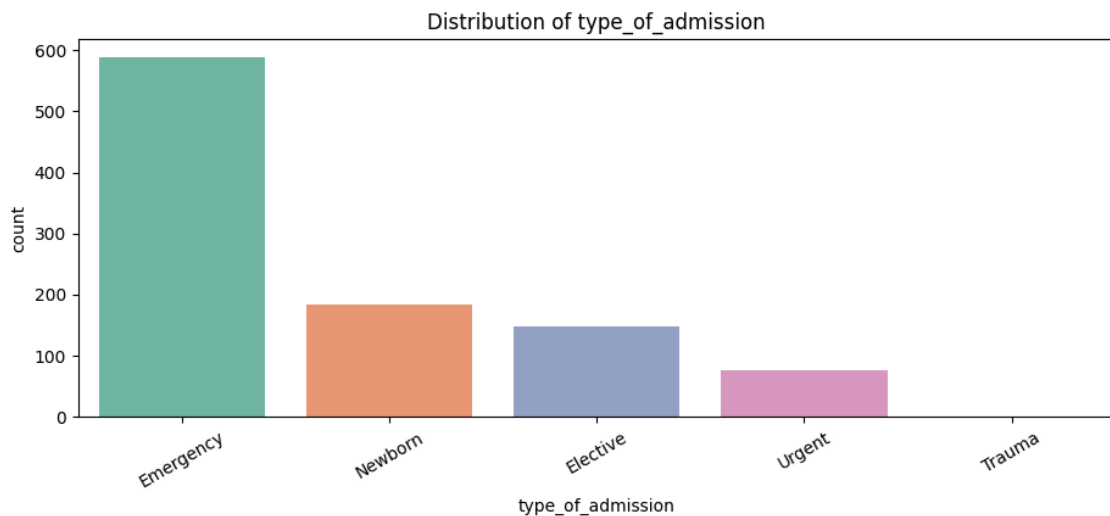
```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
    sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

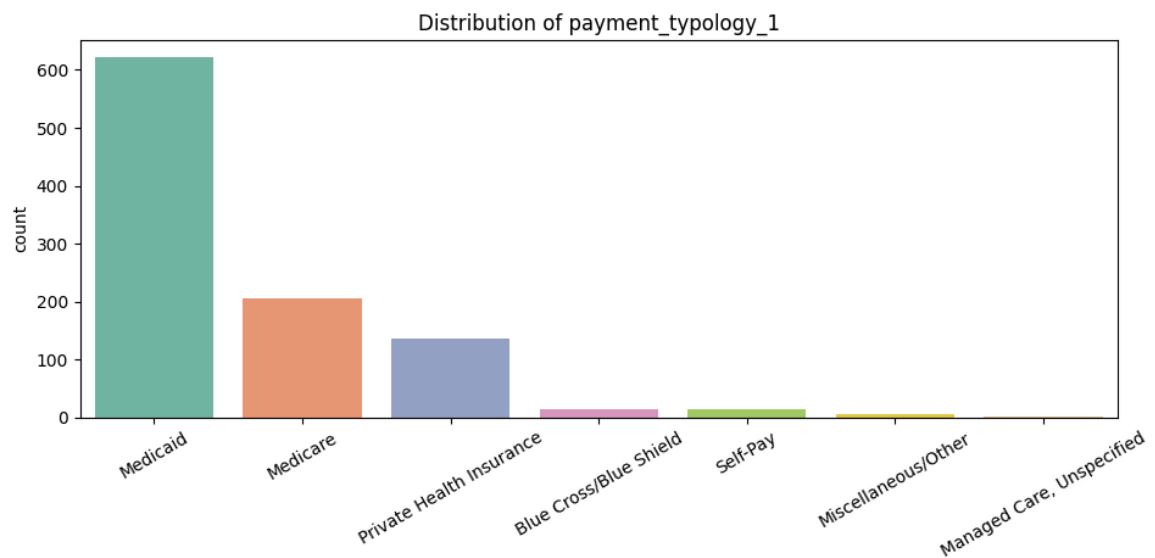
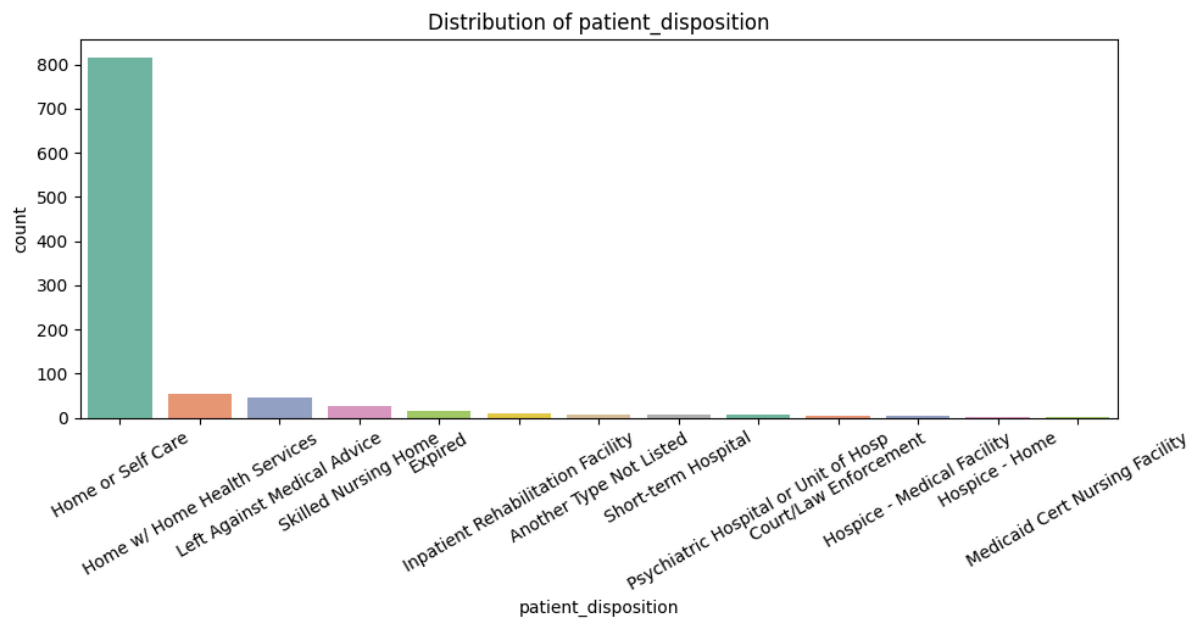
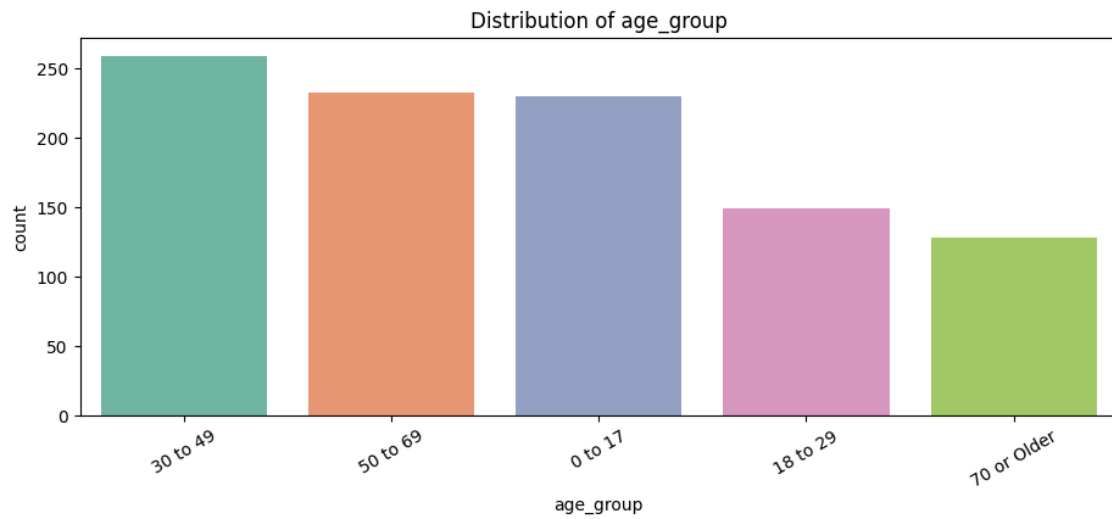

/tmp/ipython-input-19261604.py:4: FutureWarning:

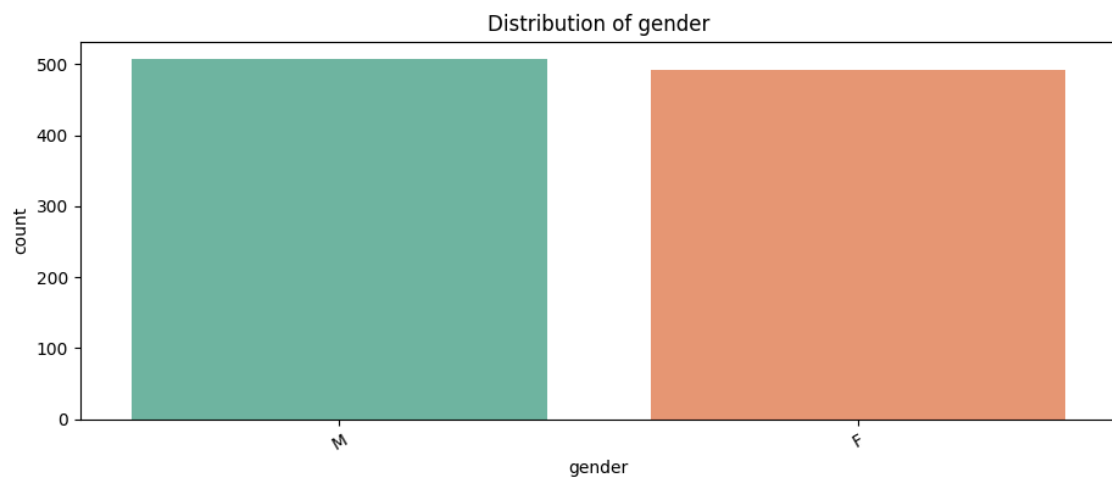
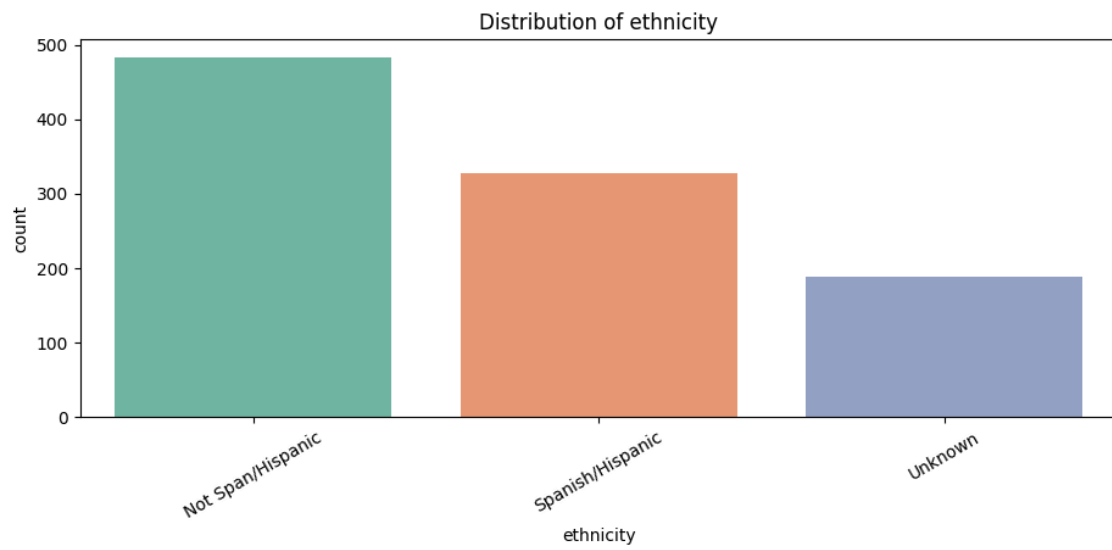
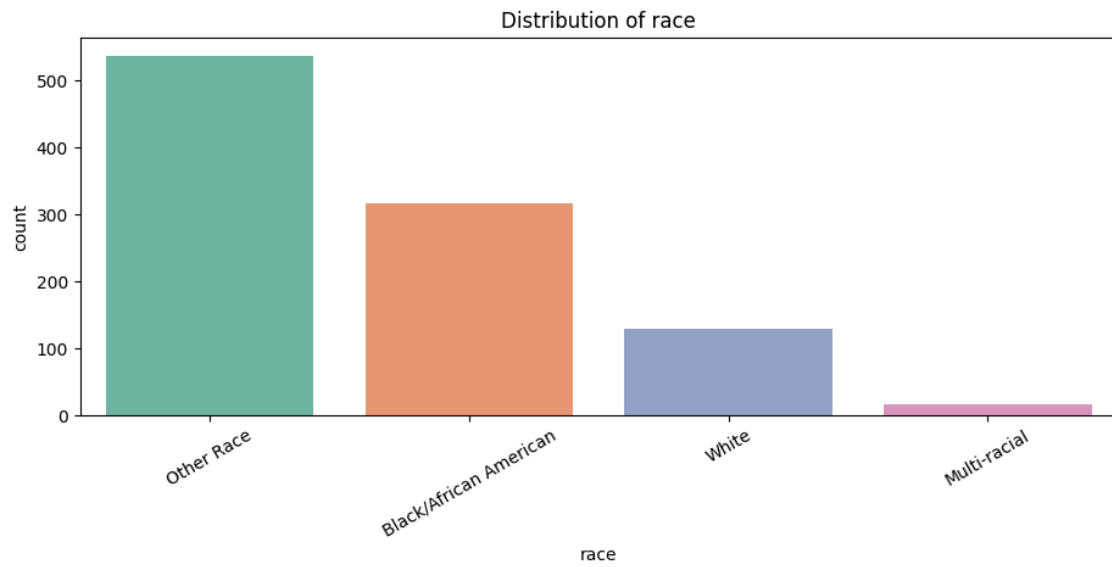
Passing `palette` without assigning `hue` is deprecated and will be removed in

```
sns.countplot(x=col, data=df, order=df[col].value_counts().index, palette="S  
/tmp/ipython-input-19261604.py:5: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```



apr_risk_of_mortality





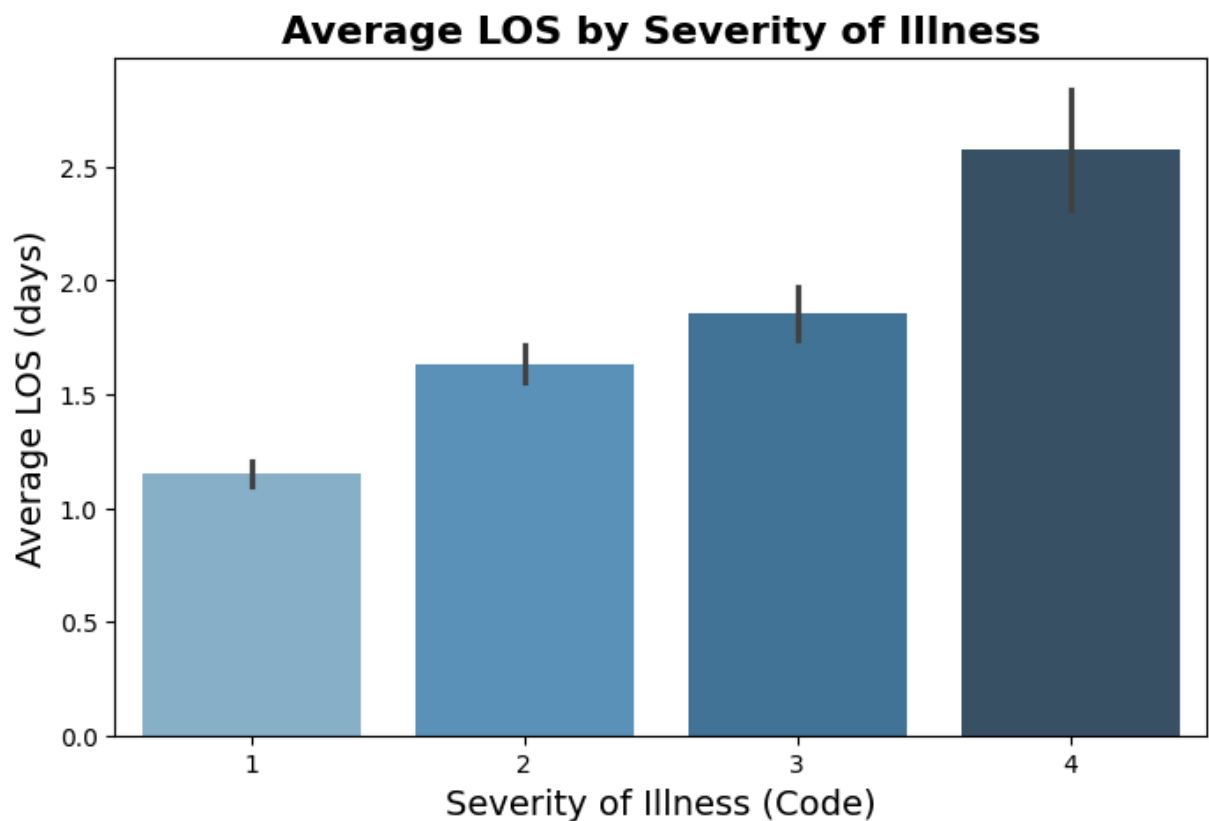
✓ Multivariates Numerical Analysis

```
# --- Barplot: Average LOS by severity ---
plt.figure(figsize=(8,5))
sns.barplot(
    x="apr_severity_of_illness_code",
    y='log_los',
    data=df,
    estimator=np.mean,
    palette="Blues_d"
)
plt.title("Average LOS by Severity of Illness", fontsize=16, weight="bold")
plt.xlabel("Severity of Illness (Code)", fontsize=14)
plt.ylabel("Average LOS (days)", fontsize=14)
plt.savefig('/content/drive/MyDrive/Colab Notebooks/Module 20 : Ensemble Techni
plt.show()
```

/tmp/ipython-input-717371407.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v

```
sns.barplot(
```



✓ Multivariates Categorical Analysis

```
fig, axes = plt.subplots(nrows=len(cat_cols), ncols=1, figsize=(12, 6 * len(cat_cols)))
for ax, col in zip(axes, cat_cols):
    sns.boxplot(x=col, y='log_los', data=df, palette="coolwarm", ax=ax)
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
    ax.set_title(f"Length of Stay by {col}", fontsize=16, weight="bold")

plt.tight_layout()
plt.savefig('/content/drive/MyDrive/Colab Notebooks/Module 20 : Ensemble Techni')
plt.show()
```



```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
sns.boxplot(x=col, y='log_loss', data=df, palette="coolwarm", ax=ax)
```

```
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
sns.boxplot(x=col, y='log_loss', data=df, palette="coolwarm", ax=ax)
```

```
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
sns.boxplot(x=col, y='log_loss', data=df, palette="coolwarm", ax=ax)
```

```
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
sns.boxplot(x=col, y='log_loss', data=df, palette="coolwarm", ax=ax)
```

```
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
sns.boxplot(x=col, y='log_loss', data=df, palette="coolwarm", ax=ax)
```

```
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
sns.boxplot(x=col, y='log_loss', data=df, palette="coolwarm", ax=ax)
```

```
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
sns.boxplot(x=col, y='log_loss', data=df, palette="coolwarm", ax=ax)
```

```
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
```

```
sns.boxplot(x=col, y='log_loss', data=df, palette="coolwarm", ax=ax)
```

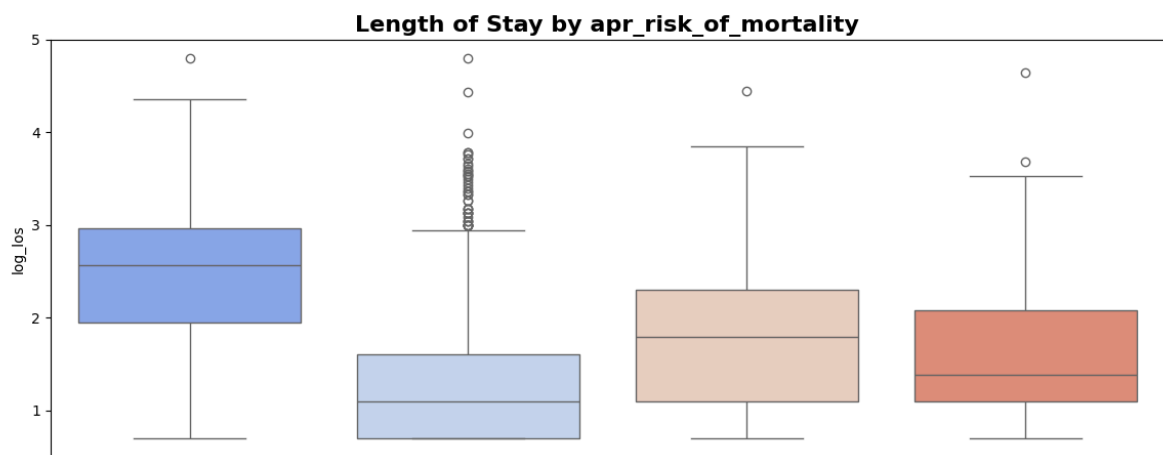
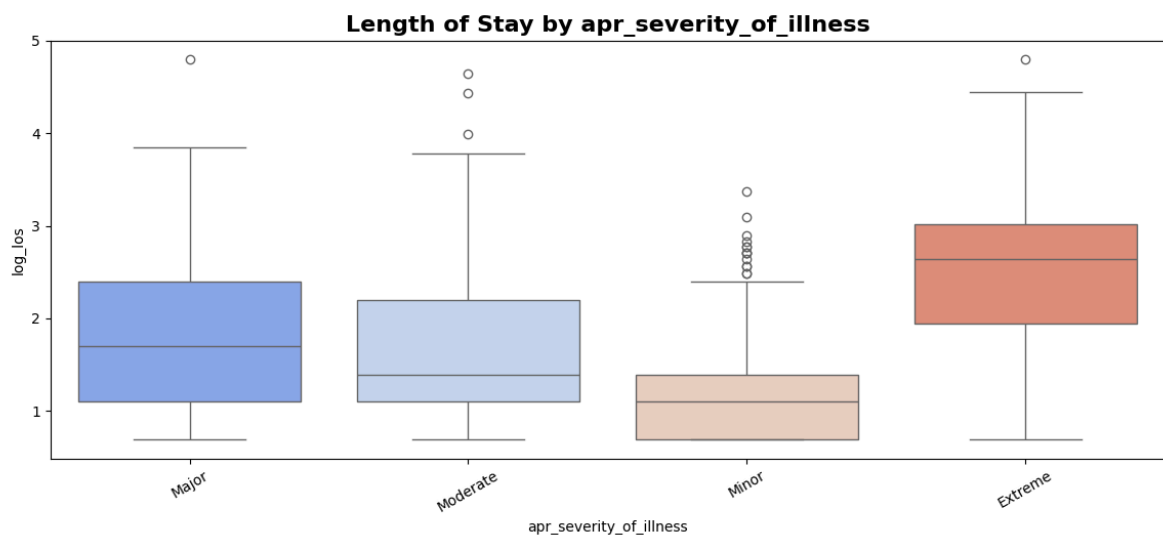
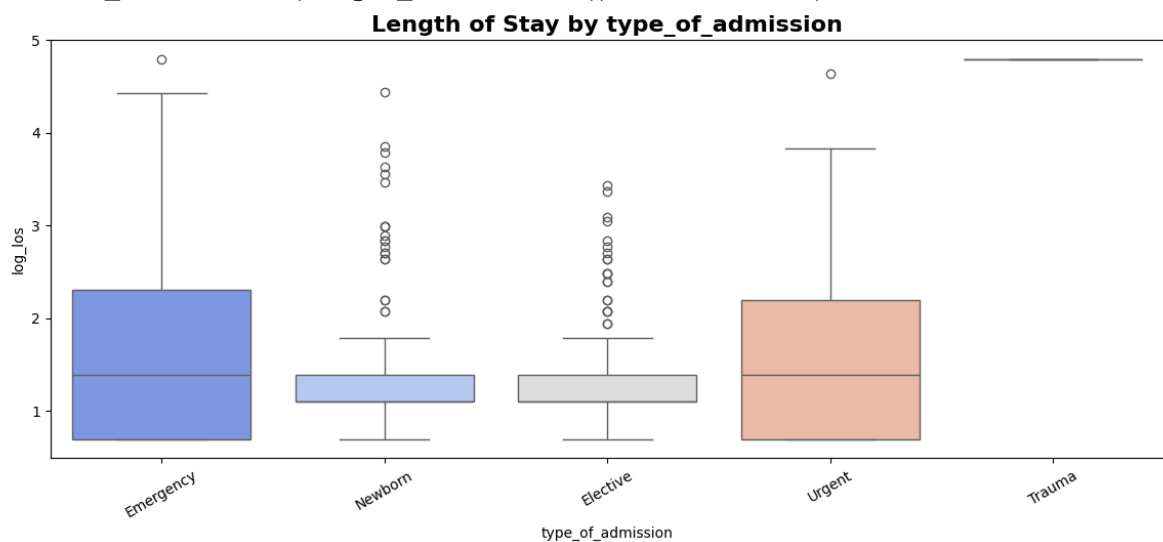
```
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be  
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```

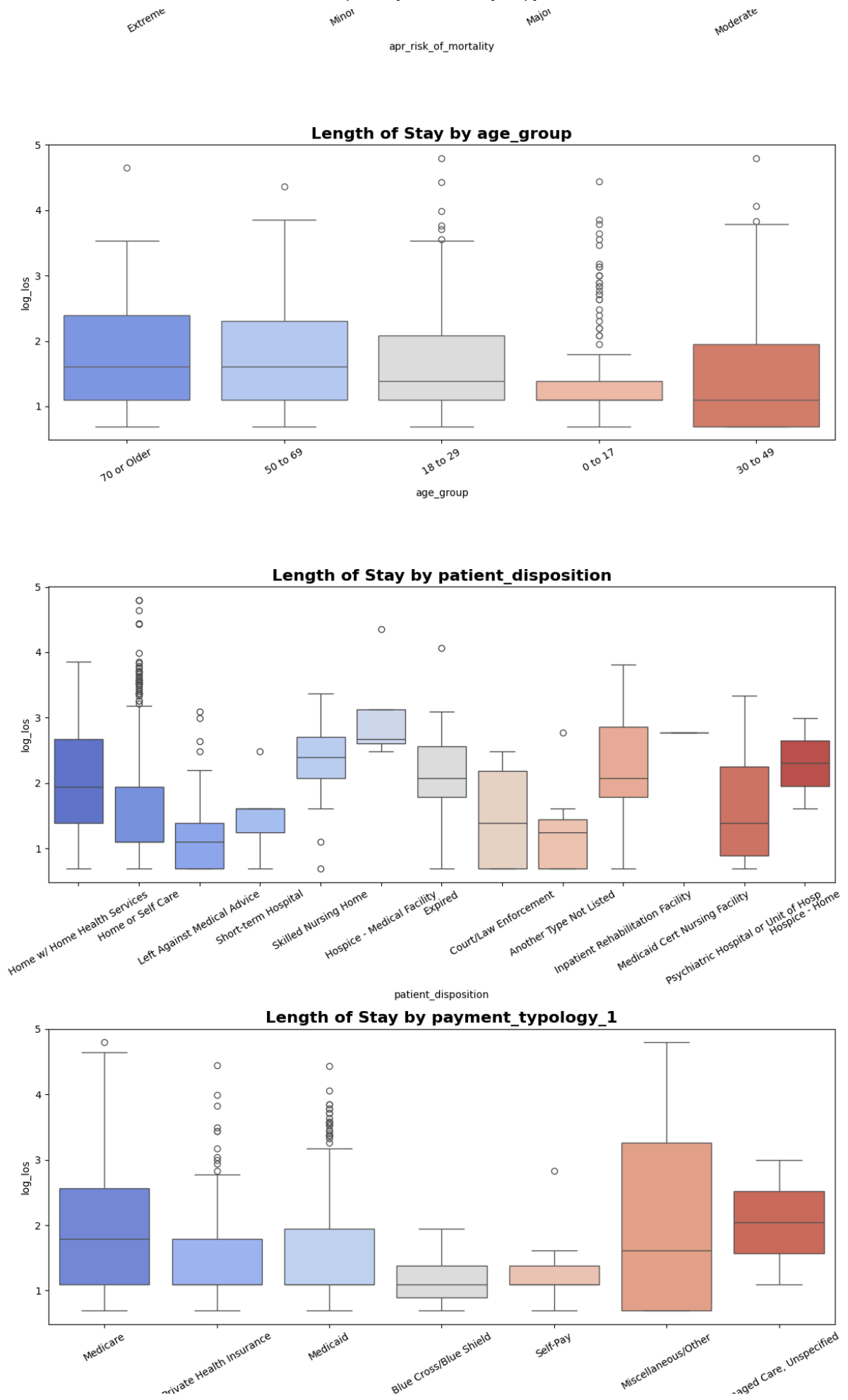


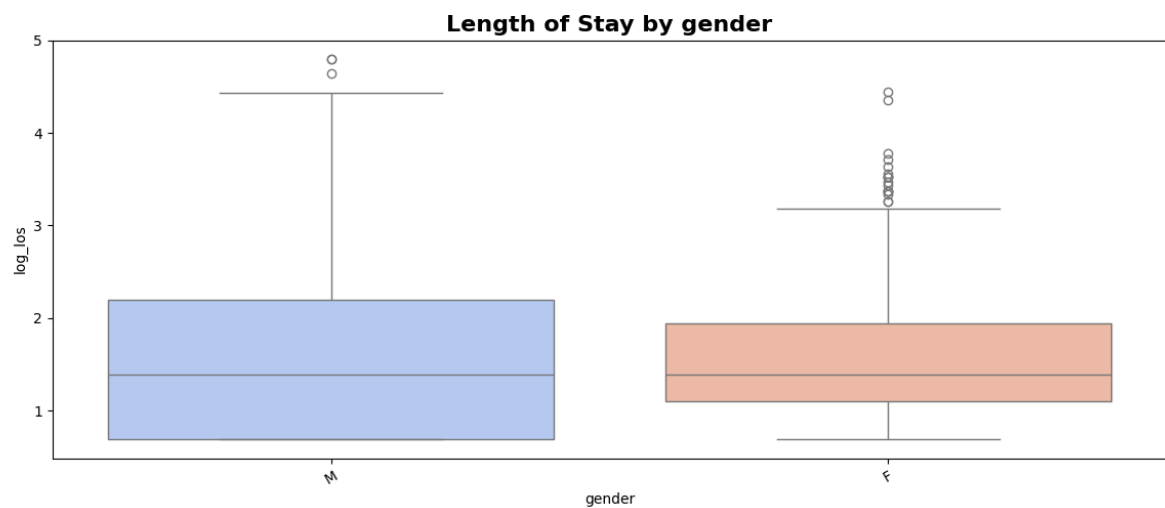
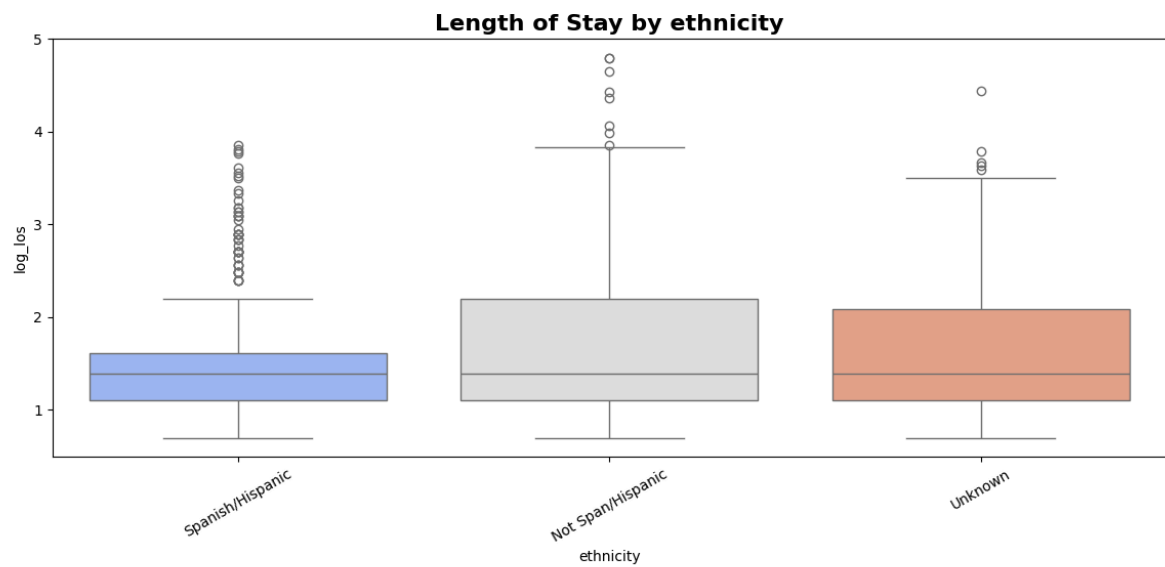
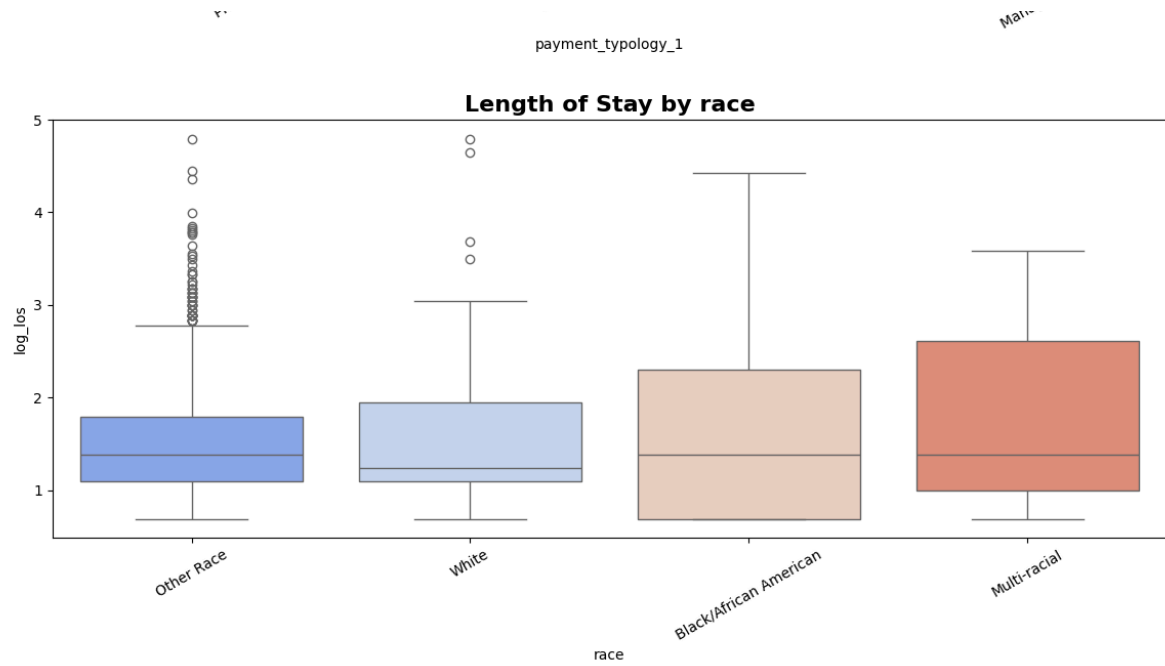
```
/tmp/ipython-input-464475691.py:3: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in

```
sns.boxplot(x=col, y='log_los', data=df, palette="coolwarm", ax=ax)
/tmp/ipython-input-464475691.py:4: UserWarning: set_ticklabels() should only be used on a single axis
ax.set_xticklabels(ax.get_xticklabels(), rotation=30)
```







[link text](#)

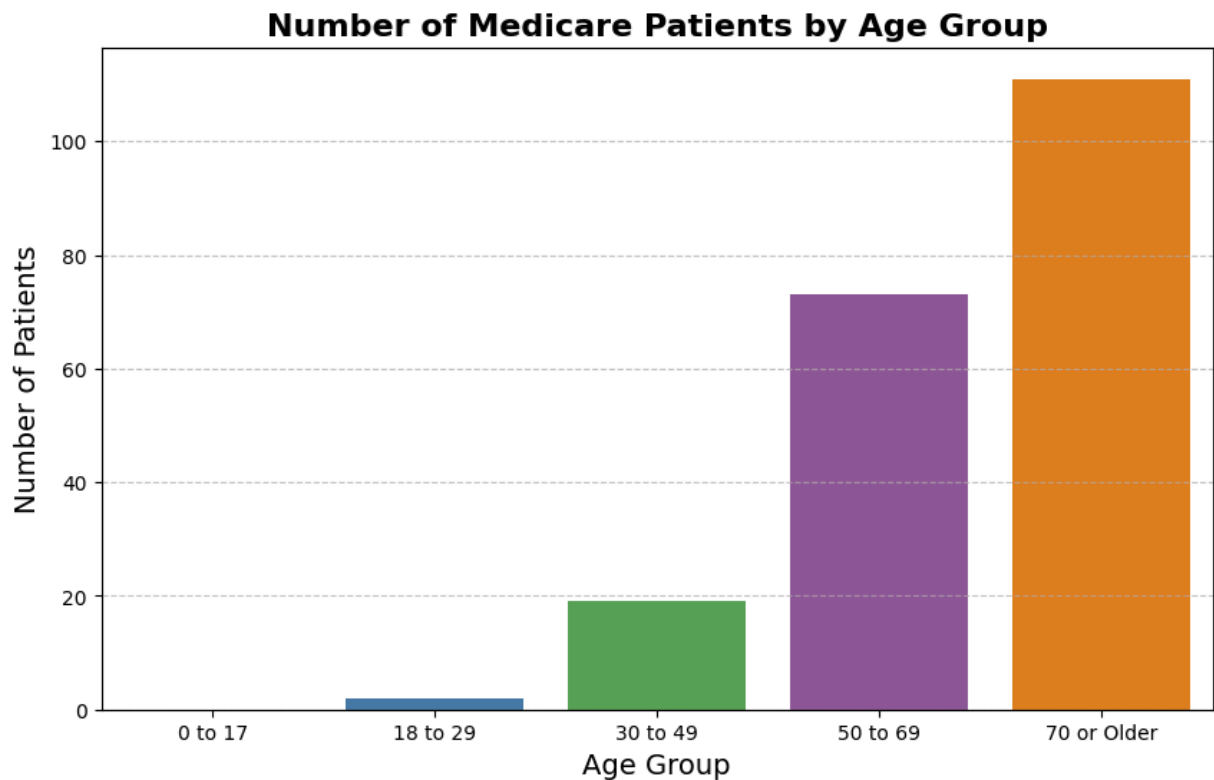
```
# Filter Medicare patients
medicare_df = df[df['payment_typology_1'] == 'Medicare']

# Create plot
plt.figure(figsize=(10,6))
sns.countplot(
    x="age_group",
    data=medicare_df,
    order=['0 to 17', '18 to 29', '30 to 49', '50 to 69', '70 or Older'],
    palette="Set1"
)
plt.title("Number of Medicare Patients by Age Group", fontsize=16, weight="bold")
plt.xlabel("Age Group", fontsize=14)
plt.ylabel("Number of Patients", fontsize=14)
plt.xticks(rotation=0)
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.savefig('/content/drive/MyDrive/Colab Notebooks/Module 20 : Ensemble Techni
plt.show()
```

```
/tmp/ipython-input-3259592023.py:6: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v
```

```
sns.countplot(
```



✓ Distribution Analysis

Numerical Features

Length of Stay (LOS):

- Distribution is right-skewed, with most patients staying 2–7 days.
- Median is 3 days, but a few outliers extend up to 120 days
- Capping or log-transforming LOS could help stabilize modeling.

Severity of Illness Code (numeric 1–4):

- LOS increases stepwise with the severity code.
- Acts as an ordinal variable and aligns well with the categorical severity labels.

Categorical Features

Type of Admission:

- Emergency admissions dominate and show longer LOS on average.
- Elective admissions tend to be shorter, reflecting planned care.
- Newborn admissions are short but with some outliers.

Severity of Illness:

- Extreme severity patients have the longest median LOS.
- Clear gradient: Minor → Moderate → Major → Extreme.
- One of the strongest predictors of LOS.

Risk of Mortality:

- Higher mortality risk correlates with longer LOS, though extreme-risk patients show greater variability (some very short stays due to death).
- Strong complement to severity.

Age Group:

- Older patients (especially 70+) stay significantly longer.
- Younger groups (0–17, 18–29) show shorter stays.
- Age amplifies Medicare's effect on LOS.

Patient Disposition:

- Home discharges have the shortest stays.
- Transfers to rehab/skilled nursing show much longer LOS.
- Expired patients show a split: some very short stays, others very long.

Payment Typology 1:

- Medicare patients have the longest stays, linked to age and comorbidities.
- Private insurance/self-pay generally result in shorter LOS.
- Reflects both demographic and system-level influences.

Gender

- The counts for "M" (male) and "F" (female) are very similar, each just under or above 500. This indicates a balanced representation of gender in the sampled data

Race

- "Other Race" is the largest group, significantly higher than others, followed by "Black/African American."
- "White" and "Multi-racial" are much less frequent — "Multi-racial" is especially rare in this sample.

Ethnicity

- "Spanish/Hispanic" is the most prominent group, with "Non-Hispanic" and "Unknown" following. "Unknown" still holds a measured portion, highlighting some missing/undisclosed data

Interaction Effects

- Severity × Type of Admission: Emergency + High severity → longest LOS.
- Age × Payment Type: Medicare patients in 70+ group drive much of the LOS burden.
- Disposition × Severity: Severe cases that transition to rehab/nursing stay longest.

Bivariate Analysis

Type of Admission vs Length of Stay

- Emergency admissions dominate the sample and are associated with longer lengths of stay on average.
- Elective admissions tend to be shorter, reflecting planned care.
- Newborn admissions are short, but with some outliers.

Severity of Illness vs Length of Stay

- Length of stay increases stepwise with the severity code (Minor, Moderate, Major, Extreme).
- Patients with "Extreme" severity have the longest median stays.
- Severity of illness is a strong predictor of length of stay.

Risk of Mortality vs Length of Stay

- Higher risk of mortality correlates with longer stays.
- "Extreme" risk patients show greater variability; some have very short stays (often due to mortality), others longer.

Age Group vs Length of Stay

- Older patients, especially those aged 70 or older, stay significantly longer in the hospital.
- Younger groups (0–17, 18–29) have the shortest stays.

Patient Disposition vs Length of Stay

- Patients discharged to home have the shortest stays.
- Transfers to rehab or skilled nursing show much longer hospital stays.
- Expired patients show a split: some have very short stays, others much longer, revealing heterogeneity in end-of-life care.

Payment Typology vs Length of Stay

- Medicare patients have the longest stays; associated with older age and complex conditions.
- Private insurance and self-pay result in shorter stays.

Gender

- There is no significant difference in the distribution of length of hospital stay between males and females, suggesting that gender is not strongly associated with hospital utilization in this sample

Race

- "Other Race" is the predominant group, followed by "Black/African American." "White" and "Multi-racial" individuals are much less represented.
- There are no drastic differences in median hospital stay across racial groups, but smaller sample sizes for less-represented races may limit the reliability of group comparisons.
- The large "Other Race" group suggests broad classification, which might mask subgroup trends and should be interpreted with caution

Ethnicity

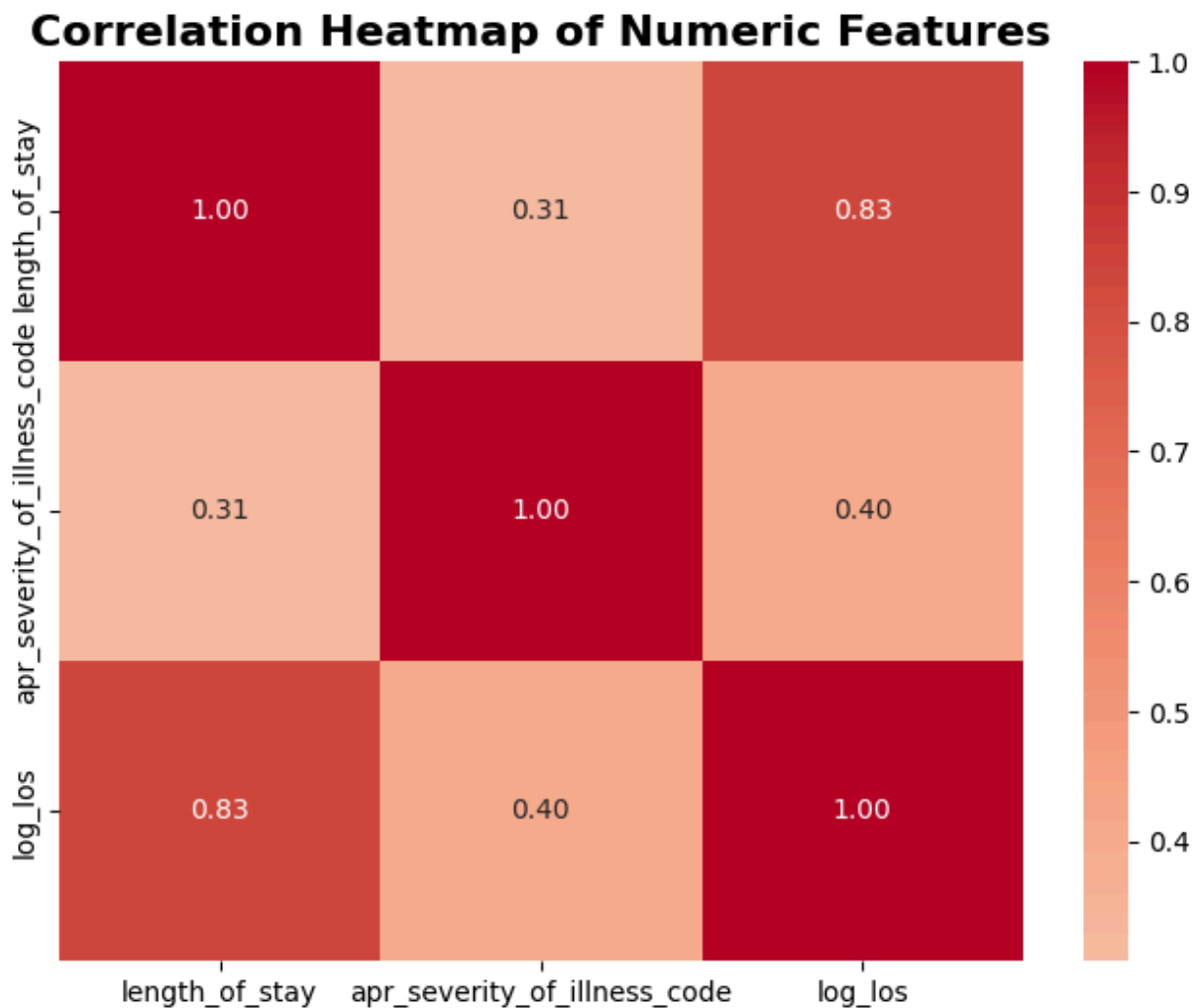
- "Spanish/Hispanic" and "Non-Hispanic" are the major categories, with "Unknown" also present in substantial amounts.

- Distribution of length of stay is similar for both main ethnicity groups, but "Unknown" has broader variability, possibly due to missing or undisclosed data.
- Ethnicity is moderately well-captured, but the significant "Unknown" count means some trends may be ambiguous or less interpretable.

✓ Correlation Map

```
# Compute correlation matrix
num_features = ['length_of_stay', 'apr_severity_of_illness_code', 'log LOS']
corr_matrix = df[num_features].corr()

# Plot heatmap
plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", center=0, fmt=".2f")
plt.title("Correlation Heatmap of Numeric Features", fontsize=16, weight="bold")
plt.savefig('/content/drive/MyDrive/Colab Notebooks/Module 20 : Ensemble Techni
plt.show()
```



Correlation Heatmap of Numeric Features —

Interpretation

The heatmap shows a moderate positive correlation (0.40) between length of stay(log) and severity of illness, indicating that patients with higher severity scores generally stay longer, though other clinical and demographic factors also play an important role in predicting LOS.

Engineering Features

```
features = [  
    'age_group', 'gender', 'race', 'ethnicity',  
    'type_of_admission', 'patient_disposition',  
    'apr_severity_of_illness', 'apr_risk_of_mortality', 'payment_typology_1'  
]  
X = df[features]  
y = df['log_los']
```

```
df['log_los'].value_counts()
```


	count
log_los	
0.693147	238
1.098612	233
1.386294	135
1.609438	64
1.791759	37
1.945910	32
2.079442	30
2.197225	25
2.708050	22
2.484907	18
2.639057	14
2.302585	13
2.397895	12
2.995732	12
2.772589	11
2.564949	10
2.890372	9
2.833213	9
3.044522	8
3.135494	6
2.944439	5

```
# Identify categorical and numerical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
numerical_cols = X.select_dtypes(exclude=['object']).columns.tolist()
```

3.178054 4
3.258097 3

✓ Train/Test Split

3.332205 3

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
```

3.713572 2

✓ A Baseline Model

```
from sklearn.dummy import DummyRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Initialize Dummy Regressor - predicts mean of target
dummy_reg = DummyRegressor(strategy='mean')
dummy_reg.fit(X_train, y_train)
```

▼ DummyRegressor ⓘ ?

DummyRegressor()

4.430617 1

```
# Predict on test data
y_pred = dummy_reg.predict(X_test)
```

```
# Evaluate
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Dummy Regressor Mean Squared Error: {mse}")
print(f"Dummy Regressor Mean Absolute Error: {mae}")
print(f"Dummy Regressor R-squared: {r2}")
```

4.430617 1
 Dummy Regressor Mean Squared Error: 0.6154251245963713
 Dummy Regressor Mean Absolute Error: 0.6536291580376619
 Dummy Regressor R-squared: -0.0006932908686412276

The dummy regressor, which always predicts the mean LOS (~6.5 days), still provides the weakest baseline. It achieves a moderate MAE due to the dataset's skew toward short stays but completely fails on extreme cases. As expected, the R^2 remains near zero, confirming that it does not capture any meaningful variance. This baseline underscores the need for incorporating meaningful clinical and demographic features.

✓ A Simple Model

```
# Identify categorical and numerical columns based on the features in X
numerical_cols = X.select_dtypes(include=['float64', 'int64']).columns.tolist()
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
```

```
# Create preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ])

```

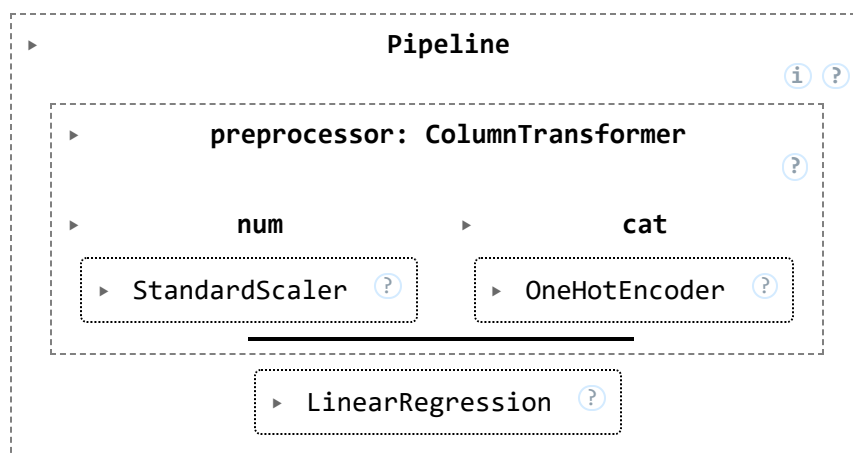
```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

# Create full pipeline with linear regression
line_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

```

```
# Train model
line_pipeline.fit(X_train, y_train)

```



```
# Predict on test set
y_pred = line_pipeline.predict(X_test)

```

Double-click (or enter) to edit

✓ Score the Model

```
# Evaluate

# Metrics

```

```

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Linear Regression Performance:")
print(f" Mean Squared Error (MSE): {mse:.2f}")
print(f" Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f" Mean Absolute Error (MAE): {mae:.2f}")
print(f" R2 Score: {r2:.3f}")

```

```

Linear Regression Performance:
Mean Squared Error (MSE): 0.48
Root Mean Squared Error (RMSE): 0.69
Mean Absolute Error (MAE): 0.54
R2 Score: 0.223

```

Linear Model Interpretation

The average error is reduced to about 4–5 days, which, while better than the raw baseline, is still large compared to the typical hospital stay (2–7 days).

The R^2 improves slightly but remains low, showing that the model explains only a limited portion of the variation in LOS.

This performance reflects the fact that, even after log transformation, LOS is driven by non-linear and interaction effects (e.g., trauma admissions, hospice transfers, and extreme illness severity) that a simple linear model cannot fully capture.

✓ Top Features with Linear (Baseline Model)

```

# Get feature names after one-hot encoding
ohe = line_pipeline.named_steps['preprocessor'].named_transformers_['cat']
encoded_cat_features = ohe.get_feature_names_out(categorical_cols)
all_features = np.concatenate([numerical_cols, encoded_cat_features])

# Get coefficients from linear model
coefs = line_pipeline.named_steps['regressor'].coef_

# Create dataframe of features and coefficient magnitudes
feat_coef_df = pd.DataFrame({
    'feature': all_features,
    'coef_magnitude': np.abs(coefs),
    'coefficient': coefs
}).sort_values(by='coef_magnitude', ascending=False)

```